

services\analysis\severity\_analyzer.py

```
# Type hints for function return structures
from typing import Dict, Any
# Efficient counting data structure for frequency analysis
from collections import Counter
# API client service for retrieving recent vulnerability data
from services.data.api_client import api_client

def get_severity_card_counts() -> Dict[str, Any]:
    """Get severity counts for dashboard cards - API ONLY"""
    print("[Severity Cards] Calculating severity distribution from API...")

    # Get recent vulnerability data from API (last 30 days)
    cves = api_client.get_cves_last_30_days()

    # Count occurrences by severity level using efficient Counter
    severity_counts = Counter()
    for cve in cves:
        # Extract severity with fallback to UNKNOWN
        severity = cve.get('Severity', 'UNKNOWN').upper()
        # Validate against known CVSS severity levels
        if severity in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW']:
            severity_counts[severity] += 1
        else:
            # Handle invalid or missing severities as UNKNOWN
            severity_counts['UNKNOWN'] += 1

    # Build comprehensive statistics structure
    result = {
        'CRITICAL': severity_counts.get('CRITICAL', 0),
        'HIGH': severity_counts.get('HIGH', 0),
        'MEDIUM': severity_counts.get('MEDIUM', 0),
        'LOW': severity_counts.get('LOW', 0),
        'UNKNOWN': severity_counts.get('UNKNOWN', 0),
        'total_cves': len(cves),
        # Calculate total excluding unknown for certain analysis
        'total_without_unknown': (severity_counts.get('CRITICAL', 0) +
            severity_counts.get('HIGH', 0) +
            severity_counts.get('MEDIUM', 0) +
            severity_counts.get('LOW', 0))
    }

    # Log detailed breakdown for monitoring and debugging
    print(f"[Severity Cards] Counts: Critical={result['CRITICAL']}, High={result['HIGH']},
    Medium={result['MEDIUM']}, Low={result['LOW']}, Unknown={result['UNKNOWN']}")
    return result

def get_severity_percentages(counts: Dict[str, Any]) -> Dict[str, str]:
    """Calculate percentages that sum to 100% using proper rounding"""
    total = counts['total_cves']
    # Handle zero division case gracefully
    if total == 0:
        return {k: "0%" for k in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW', 'UNKNOWN']}

    # Calculate exact floating-point percentages
    exact_percentages = {
        'CRITICAL': (counts['CRITICAL'] * 100 / total),
        'HIGH': (counts['HIGH'] * 100 / total),
        'MEDIUM': (counts['MEDIUM'] * 100 / total),
        'LOW': (counts['LOW'] * 100 / total),
        'UNKNOWN': (counts['UNKNOWN'] * 100 / total)
    }

    # Round each percentage to nearest integer
    rounded_percentages = {k: round(v) for k, v in exact_percentages.items()}

    # Check if rounding caused sum to deviate from 100%
    total_rounded = sum(rounded_percentages.values())
    difference = 100 - total_rounded

    # Apply largest remainder method to correct rounding errors
    if difference != 0:
        # Calculate fractional parts for adjustment priority
```

```

fractional_parts = {k: exact_percentages[k] - rounded_percentages[k]
for k in exact_percentages.keys()}

if difference > 0:
# Need to add percentage points to reach 100%
for _ in range(difference):
# Add to category with largest positive fractional part
max_key = max(fractional_parts.keys(), key=lambda k: fractional_parts[k])
rounded_percentages[max_key] += 1
fractional_parts[max_key] -= 1
else:
# Need to subtract percentage points to reach 100%
for _ in range(abs(difference)):
# Subtract from category with largest negative fractional part
min_key = min(fractional_parts.keys(), key=lambda k: fractional_parts[k])
rounded_percentages[min_key] -= 1
fractional_parts[min_key] += 1

# Convert to display-ready percentage strings
return {k: f"{v}%" for k, v in rounded_percentages.items()}

def get_severity_distribution_pie() -> Dict[str, Any]:
    """Get severity distribution for pie chart - includes UNKNOWN"""
    print("[Severity Distribution] Using severity card counts for pie chart...")

# Get base severity statistics
counts = get_severity_card_counts()
# Calculate accurate percentages that sum to 100%
percentages = get_severity_percentages(counts)

# Build complete chart data structure
result = {
'counts': counts,                # Raw count data
'percentages': percentages,      # Formatted percentage strings
'chart_data': {
# Labels for chart legend and display
'labels': ['Critical', 'High', 'Medium', 'Low', 'Unknown'],
# Values for chart segments
'values': [
counts['CRITICAL'],
counts['HIGH'],
counts['MEDIUM'],
counts['LOW'],
counts['UNKNOWN']
],
# Colors matching severity levels (accessible color scheme)
'colors': ['#f55855', '#f8a541', '#3b8ded', '#42d392', '#6b7280']
},
'total_including_unknown': counts['total_cves']
}

print(f"[Severity Distribution] Pie chart data generated with
{result['total_including_unknown']} total CVEs")
return result

```